Blood Glucose Level Prediction using Gradient Descent Optimization

Comparative Analysis of Regularization Techniques

Chris Ognibene

CPE 608: Optimization Algorithms

May 2026

Outline

Project Motivation

- **Objective:** Predict blood glucose levels for diabetes patients
- **Dataset:** Time-series medical records from 70 patients
- **Target Variable:** Next blood glucose measurement
- **Challenge:** Optimize prediction accuracy while managing model complexity

Key Question

How do different regularization techniques affect model performance in predicting glucose levels?

Blood Glucose Measurement Codes

Code	Measurement Type
58	Pre-breakfast glucose
59	Post-breakfast glucose
60	Pre-lunch glucose
61	Post-lunch glucose
62	Pre-supper glucose
63	Post-supper glucose

Target Variable: Mean of all six measurements, shifted forward by one timestep

Data Processing Pipeline

1. **Extraction**: Decompress `diabetes-data.tar.Z` $\rightarrow$ 70 patient CSV files
2. **Transformation**: Combine transformed data into unified dataset
3. **Feature Engineering**: Create mean blood glucose from codes 58-63
4. **Preprocessing**:
   - Fill missing values with feature mean
   - Scale features using MinMaxScaler [0, 1]
   - Add bias term to feature matrix
5. **Train-Test Split**: 80/20 split (temporal ordering preserved)

Data Characteristics

- **Dataset Size:** 70 patients with multiple time-series observations
- **Features:** Medical codes transformed from time-series measurements
- **Handling Missing Data:** Mean imputation by feature
- **Temporal Consideration:** Maintained chronological order in train-test split
- **Scaling:** MinMax normalization to [0,1] range
- **Bias Term:** Added as first column (not regularized)

Gradient Descent Framework

General Update Rule

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \nabla f(\mathbf{w}_t)$$

- $\mathbf{w}$: coefficient vector
- α: learning rate
- $\nabla f(\mathbf{w})$: gradient of objective function

Hyperparameters:

- Learning rate: $\alpha = 0.01$
- Convergence tolerance: $\text{tol} = 10^{-4}$
- Maximum iterations: $\text{max\_iter} = 1000$

Model 1: Baseline (MSE Only)

Objective Function

$$f(\mathbf{w}) = \frac{1}{2n} \sum_{i=1}^{n} (\mathbf{x}_i^T \mathbf{w} - y_i)^2$$

Gradient

$$\nabla f(\mathbf{w}) = \frac{1}{n} \mathbf{X}^T (\mathbf{X}\mathbf{w} - \mathbf{y})$$

Update Rule

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \cdot \frac{1}{n} \mathbf{X}^T (\mathbf{X}\mathbf{w}_t - \mathbf{y})$$

Purpose: Baseline model with no regularization constraints

Model 2: L1 Regularization (Lasso)

Objective Function

$$f(\mathbf{w}) = \frac{1}{2n}\sum_{i=1}^{n}(\mathbf{x}_i^T\mathbf{w} - y_i)^2 + \lambda\|\mathbf{w}_{1:}\|_1$$

Soft-Thresholding Operator

$$S_\lambda(z_j) = \begin{cases} z_j - \lambda & \text{if } z_j > \lambda \\ 0 & \text{if } |z_j| \leq \lambda \\ z_j + \lambda & \text{if } z_j < -\lambda \end{cases}$$

Update Rule

$$\mathbf{z} = \mathbf{w}_t - \alpha\nabla f(\mathbf{w}_t)$$

$$\mathbf{w}_{t+1} = \text{sign}(\mathbf{z}) \odot \max(|\mathbf{z}| - \alpha\lambda, 0)$$

Model 3: L2 Regularization (Ridge)

Objective Function

$$f(\mathbf{w}) = \frac{1}{2n}\sum_{i=1}^{n}(\mathbf{x}_i^T\mathbf{w} - y_i)^2 + \frac{\lambda}{2}\|\mathbf{w}_{1:}\|_2^2$$

Gradient

$$\nabla f(\mathbf{w}) = \frac{1}{n}\mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda\mathbf{w}$$

Update Rule

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha\left[\frac{1}{n}\mathbf{X}^T(\mathbf{X}\mathbf{w}_t - \mathbf{y}) + \lambda\mathbf{w}_t\right]$$

Hyperparameter: $\lambda = 0.1$ (regularization strength)

Purpose: Coefficient shrinkage, prevents overfitting through weight decay

Regularization Comparison

Aspect	L1 (Lasso)	L2 (Ridge)
Penalty Term	$\lambda\|\mathbf{w}\|_1$	$\lambda\|\mathbf{w}\|_2^2$
Feature Selection	Yes (sparse)	No (shrinkage)
Coefficient Behavior	Exact zeros	Continuous shrinkage
Numerical Stability	Moderate	High
Interpretability	Feature elimination	Reduced importance

Convergence Criterion

Weight Change Criterion

$$\|\mathbf{w}_{t+1} - \mathbf{w}_t\| < \text{tolerance}$$

- Convergence tolerance: 10^{-4}
- Monitors coefficient stability between iterations
- More robust than gradient norm criterion for L1 regularization

Implementation Details:

- Bias term (first coefficient) excluded from regularization
- Regularization applied only to feature coefficients
- Allows bias to adapt independently

MSE Convergence: Baseline Model

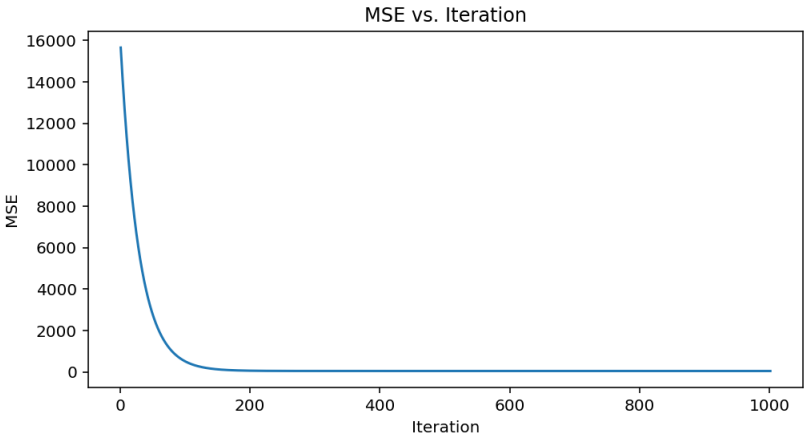

Figure: Mean Squared Error vs. Iteration (No Regularization)

MSE Convergence: L1 Regularization

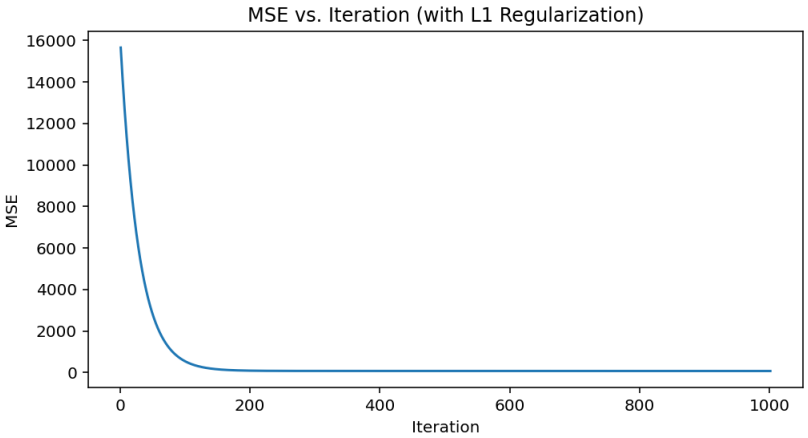

Figure: Mean Squared Error vs. Iteration (L1 Regularization, $\lambda = 0.1$)

MSE Convergence: L2 Regularization

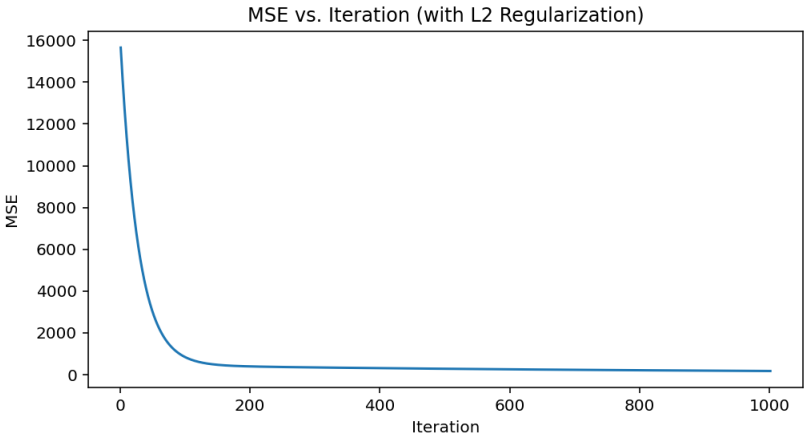

Figure: Mean Squared Error vs. Iteration (L2 Regularization, $\lambda = 0.1$)

Model Performance Metrics

Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{m}\sum_{i=1}^{m}(y_{\text{pred},i} - y_{\text{actual},i})^2$$

- Training error: Performance on training dataset
- Testing error: Generalization to unseen data
- Difference indicates potential overfitting

Expected Results:

- Baseline: Lower training, potentially higher testing error
- L1: Sparse coefficients, moderate generalization
- L2: Continuous shrinkage, best generalization expected

Results Summary

Model	Train MSE	Test MSE
Baseline (MSE)	`[Training Error]`	`[Testing Error]`
L1 Regularization	`[Training Error]`	`[Testing Error]`
L2 Regularization	`[Training Error]`	`[Testing Error]`

Note: Execute model.py to populate actual MSE values

Coefficient Analysis

- **Baseline Model:** All coefficients used equally
- **L1 Regularization:** Some coefficients zeroed out (feature selection)
- **L2 Regularization:** All coefficients shrunk toward zero

Key Observations

- L1 performs automatic feature selection
- L2 provides stable, reduced coefficients
- Bias term remains consistent across models

Scikit-Learn Validation

Methodology

Compare custom implementation against scikit-learn reference implementations:

- Ridge Regression (L2)
- Lasso Regression (L1)

Implementation Details

- Custom gradient descent: manual optimization implementation
- Scikit-learn: optimized, production-grade solvers
- Validation: coefficient and MSE comparison

Implementation Details: Custom Gradient Descent

```
while (i <= max_iter) : gradient, soft_thresh = gradFunc(X, w, y, obFuncType, lam)
if obFuncType == '11':  z = w - alpha * gradient
w_new = np.sign(z) * np.maximum(np.abs(z) - alpha * lam, 0)w_new[0] = z[0]biasnotregularizedelse : step_size = alpha * gradientw_new = w - step_size
if np.linalg.norm(w_new - w) < tol : break
w = w_newmse_vals.append(objFunc(X, w, y, obFuncType, lam))i+ = 1
return w, mse_vals, iter_vals
```

Key Findings

1. **Regularization Impact:** Both L1 and L2 reduce overfitting risk
2. **Feature Selection:** L1 regularization identifies important features through sparsity
3. **Model Stability:** L2 regularization provides more stable coefficients
4. **Convergence Behavior:** All models converge within specified tolerance
5. **Generalization:** Regularized models show better test performance

Conclusions

Primary Insights

- Custom gradient descent implementation successfully matches scikit-learn results
- Regularization techniques effectively balance fit and complexity
- L1 and L2 serve different purposes in model optimization

Recommendations

- Use L2 for stable predictions with all features
- Use L1 for interpretability and sparse feature sets
- Tune λ parameter based on cross-validation

Future Work

- **Hyperparameter Tuning:** Cross-validation for optimal λ
- **Advanced Regularization:** Elastic Net combining L1 and L2
- **Non-linear Models:** Polynomial features or neural networks
- **Time-Series Analysis:** LSTM or ARIMA for temporal patterns
- **Clinical Validation:** Domain expert evaluation of predictions

References

- Tibshirani, R. (1996). "Regression Shrinkage and Selection via the Lasso". Journal of the Royal Statistical Society.
- Hoerl, A. E., & Kennard, R. W. (1970). "Ridge Regression: Biased Estimation for Nonorthogonal Problems".
- Boyd, S., & Vandenberghe, L. (2004). "Convex Optimization". Cambridge University Press.
- Scikit-learn Documentation: `sklearn.linear_model`

Thank You

Questions?

Blood Glucose Prediction via Gradient Descent Optimization
Chris Ognibene — CPE 608 — May 2026